

Students App Lab

Lab Day 7 | DOM Manipulation with JavaScript

Open **students-app.html** in the browser, write your code inside the `<script>` tag (or a separate JS file), and follow the steps below in order. Each step includes a Hint to guide you without giving away the full solution.

STEP 1

Select the HTML elements

Grab every element you will need to work with, using their id. You need: the name input, the grade input, the add button, the table body (tbody), and the error box.

Hint:

```
document.getElementById("theId")

var addBtn = document.getElementById("addBtn");
```

STEP 2

Set up your data

Create an empty array to hold all students, and a counter variable to give each new student a unique id.

Hint:

```
var students = [];
var nextId = 1;
```

STEP 3

Create a function that builds a student object

Write `createStudent(id, name, grade)` that returns an object with `id`, `name`, `grade`, and a method `getStatus()` that returns "Pass" if `grade >= 50`, otherwise "Fail".

Hint:

```
function createStudent(id, name, grade) {
  return {
    id: id,
    name: name,
    grade: grade,
    getStatus: function () {
      return this.grade >= 50 ? "Pass" : "Fail";
    }
  };
}
```

STEP 4

Add a click listener on the "Add" button

This is where everything connects together.

Hint:

```
addBtn.addEventListener("click", function () {  
  // ...  
});
```

STEP 5 Read the values typed by the user

■ Tricky part — read this hint carefully

Inside the click listener, you need to grab what the user typed in the name input and the grade input.

Hint:

```
// Every input element has a ".value" property  
// that ALWAYS returns a STRING  
// (even if the input type is "number")  
  
var name = nameInput.value; // e.g. "Ahmed"  
var gradeText = gradeInput.value; // e.g. "85" (still text!)  
  
// Since gradeText is a string, use Number(gradeText)  
// to turn it into an actual number before doing  
// any math checks on it.
```

STEP 6 Validate the input

Check that: name is not empty, grade is a valid number, and grade is between 0 and 100. If something is wrong, show a message inside #errorBox and STOP (don't create the student).

Hint:

```
if (name.trim() === "") {  
  errorBox.innerText = "Name cannot be empty";  
  return;  
}  
var grade = Number(gradeText);  
if (isNaN(grade)) { /* ... */ }
```

STEP 7 Create the student and store it

Use createStudent() from Step 3, push it into the students array, then increase nextId by 1.

Hint:

```
var student = createStudent(nextId, name, grade);  
students.push(student);  
nextId++;
```

STEP 8 Build the table row (DOM creation)

Write renderStudentRow(student): create a <tr>, call student.getStatus() to know Pass/Fail, add classList.add("pass") or classList.add("fail"), build tr.innerHTML with 4 <td> cells (id, name, grade, status),

then append it to tbody.

Hint:

```
var tr = document.createElement("tr");

tr.innerHTML = "<td>" + student.id + "</td>" +
    "<td>" + ... ;
```

✓ STEP 9 Clear the form after adding

After successfully adding a student, reset the inputs so the user can add the next one.

Hint:

```
nameInput.value = "";
gradeInput.value = "";
nameInput.focus();
```

STEP 10 (Bonus) Wrap validation in try/catch

Instead of "if wrong -> return", make validateInput() throw an Error, and wrap the whole click logic in try/catch so any error message ends up in #errorBox automatically.

Hint:

```
try {
    var grade = validateInput(name, gradeText); // may throw
    // ...
} catch (error) {
    errorBox.innerText = "Error: " + error.message;
}
```